

Arquitectura — Red Social con Microservicios

Autor: Leonel Benítez

Fecha: 2026-08-07

Plazo de ejecución: 2 días

Repositorio: <https://github.com/LeOx26/periferia-social>

Video demostrativo (4 min): <https://github.com/LeOx26/periferia-social/releases/download/v1.0.0/periferia-social-demo.mp4>

1. Contexto y criterio de diseño

El dominio es una red social mínima: autenticación, feed de publicaciones, publicar, ver perfil y dar likes en tiempo real. Cinco casos de uso simples, repartidos en microservicios Java, una web en React, una app en React Native y todo en contenedores.

El coste real del ejercicio no está en la lógica de negocio —es trivial— sino en la **infraestructura** y en la **reutilización de código entre plataformas**. Ahí es donde se concentran las decisiones que importan.

Con un plazo de dos días, el criterio ha sido:

> **Preferir la decisión pequeña y defendible sobre el volumen de código.** Cada elección de este documento tiene una alternativa considerada y una razón explícita. Donde se ha tomado un atajo, queda declarado como tal.

Consecuencia práctica: se ha recortado alcance antes que rigor. La sección 12 lista lo que queda fuera deliberadamente y la sección 11 recoge la evolución natural del sistema con más tiempo.

2. Decisiones tomadas

#	Decisión	Alternativa descartada	Razón
D1	2 microservicios: <code>auth-service</code> , <code>social-service</code>	3 servicios; 3 + API Gateway	Dos bounded contexts reales (identidad y contenido social). Más servicios solo multiplican boilerplate sin añadir separación conceptual. El gateway se come medio día y añade un punto de fallo en la demo.
D2	Bases de datos separadas (<code>authdb</code> , <code>socialdb</code>) en un mismo contenedor Postgres	Base compartida con FK entre <code>posts</code> y <code>users</code>	La base compartida es el antipatrón clásico de microservicios. Separarlas cuesta ~15 min y demuestra ownership de datos. Un contenedor en vez de dos porque el aislamiento que importa es lógico, no de proceso.
D3	Sin comunicación entre servicios. <code>social-service</code> valida el JWT localmente con el secreto compartido	Llamada HTTP a <code>auth-service</code> para validar/resolver autor	Elimina acoplamiento en runtime y un punto de fallo. El <code>alias</code> del autor viaja como claim del JWT.
D4	Alias del autor denormalizado en <code>posts.author_alias</code>	Join cruzado; llamada al otro servicio	Consecuencia directa de D2+D3. El feed no necesita ninguna consulta fuera de <code>socialdb</code> .
D5	UUIDs fijos y deterministas para los usuarios sembrados, declarados en ambos scripts de migración	Seed coordinado en runtime entre servicios	Único modo honesto de sembrar dos bases independientes. En producción sería un evento <code>UserRegistered</code> que <code>social-service</code> consume para construir su proyección de autores; queda documentado como evolución.
D6	WebSocket nativo (<code>TextWebSocketHandler</code>) sin STOMP ni SockJS	STOMP sobre SockJS	SockJS es problemático en React Native y STOMP añade una capa de broker innecesaria para un evento de forma <code>{postId, likeCount}</code> . El mismo cliente WS sirve web y móvil sin ramificaciones.
D7	Sin procedimientos almacenados PL/pgSQL	Lógica de likes en la BD	El enunciado los marca opcionales pero exige justificar. Un dominio rico (reglas en la entidad <code>Post</code>) se alinea con el extra de DDD que el propio enunciado valora, es testeable sin infraestructura y mantiene la lógica bajo control de versiones.
D8	Monorepo único	Repos separados backend/frontend	El enunciado se contradice (Objetivo dice "repositorio(s) separados", Entregables dice "repositorio con backend y frontend"). El monorepo es lo único coherente con exigir un <code>packages/core</code> compartido entre web y móvil.
D9	Feed muestra todas las publicaciones , incluidas las propias, marcadas y no likeables	Ocultar las propias (lectura literal del enunciado)	Ocultar el post que acabas de crear es confuso y perjudica la demo. El espíritu del requisito ("interactúas con otros") se hace cumplir donde

			importa: no puedes dar like a tu propia publicación.
D10	Observabilidad incluida (Actuator, Prometheus, Grafana, logs con correlation-id)	Recortarla por tiempo	Es lo que distingue un sistema operable de uno que solo funciona en la máquina del autor. Además da el mejor plano del video demo.
D11	Móvil en Simulador de iOS + Expo Web	Dispositivo físico con Expo Go	El simulador comparte <code>localhost</code> con el host (cero configuración de IP) y se graba con captura de pantalla nativa. Expo Web permite al evaluador abrir la app sin instalar Xcode.
D12	Java 21 LTS	Java 26; compilar solo dentro de Docker	Spring Boot 4.1 soporta oficialmente hasta Java 26, pero 26 no es LTS y el ecosistema de pruebas (ByteBuddy/Mockito) suele ir por detrás con JDKs recientes. Java 21 es LTS, es el objetivo más probado del ecosistema y su imagen base de Docker es ubicua. Compilar solo en Docker haría que cada iteración de tests tardara minutos en vez de segundos.
D13	Sin Lombok	Lombok para entidades y DTOs	Lombok es un procesador de anotaciones que rompe con frecuencia al cambiar de versión de JDK. Los <code>record</code> de Java cubren los DTOs sin dependencias, y las entidades JPA son pocas. Se elimina un riesgo de build a cambio de escribir unos cuantos getters.

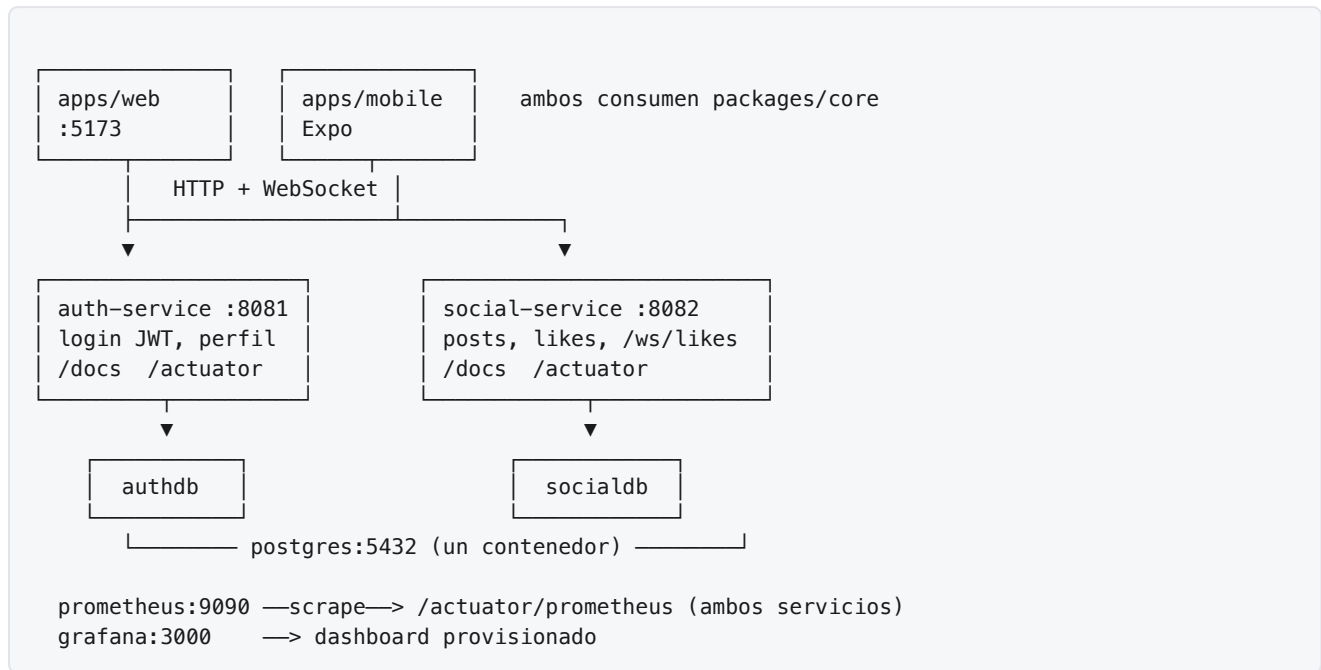
2b. Stack y versiones

Versiones verificadas contra `start.spring.io` y los metadatos de Maven Central el 2026-08-07.

Pieza	Versión	Nota
JDK	21 LTS (<code>openjdk@21</code> de Homebrew)	Convive con el Corretto 26 ya presente; se selecciona por <code>JAVA_HOME</code> . Se usa la fórmula y no el cask porque el cask instala un <code>.pkg</code> que exige privilegios de administrador.
Spring Boot	4.1.0	La rama 3.x ya no se ofrece en <code>start.spring.io</code> ; entregar sobre una versión sin soporte no sería defendible.
Build backend	Maven Wrapper (<code>mvnw</code>) por servicio	Versionado en el repo: no hace falta tener Maven instalado. Cada servicio tiene su <code>pom.xml</code> y su <code>Dockerfile</code> — son unidades desplegables independientes, no módulos de un mismo build.
springdoc-openapi	3.1.0	La serie 2.x es para Spring Boot 3.x.
jjwt	0.13.0	Firma y verificación HS256.
logstash-logback-encoder	9.0	Logs en JSON.
Imagen Docker	<code>maven:3-eclipse-temurin-21</code> (build) → <code>eclipse-temurin:21-jre</code> (runtime)	Multi-stage. Fija el JDK, así que la máquina del evaluador es irrelevante.
Node	26	Ya instalado.
Gestor de paquetes	pnpm 11 con workspaces	—
Postgres	16	—

Gotcha conocido: Expo no resuelve bien el `node_modules` simbolizado de pnpm. El repo incluye un `.npmrc` con `node-linker=hoisted` para evitar el fallo de resolución de módulos en Metro. Es una hora perdida si se descubre tarde.

3. Arquitectura



Estructura del repositorio



4. Modelo de datos

authdb

users

Columna	Tipo	Notas
id	UUID	PK
username	VARCHAR(50)	UNIQUE, NOT NULL
password_hash	VARCHAR(100)	BCrypt
first_name	VARCHAR(80)	NOT NULL
last_name	VARCHAR(80)	NOT NULL
birth_date	DATE	NOT NULL
alias	VARCHAR(30)	UNIQUE, NOT NULL
created_at	TIMESTAMPTZ	default now()

socialdb

posts

Columna	Tipo	Notas
id	UUID	PK
author_id	UUID	NOT NULL, sin FK (otro bounded context)
author_alias	VARCHAR(30)	NOT NULL, denormalizado desde el claim del JWT
message	VARCHAR(280)	NOT NULL
created_at	TIMESTAMPTZ	NOT NULL, lo fija el dominio
like_count	INT	NOT NULL default 0

Índice: idx_posts_created_at DESC (orden del feed).

post_likes

Columna	Tipo	Notas
post_id	UUID	NOT NULL, FK → posts
user_id	UUID	NOT NULL
created_at	TIMESTAMPTZ	default now()

Restricción: UNIQUE(post_id, user_id) — red de seguridad de la invariante anti-doble-like.

Migraciones y seed

Flyway en ambos servicios. El seed son migraciones SQL versionadas, lo que satisface directamente el entregable "script para la base de datos con usuarios predefinidos" sin artefactos adicionales.

`authdb: V1__schema.sql, V2__seed_users.sql` (5 usuarios, contraseña conocida documentada en el README, hashes BCrypt precalculados)

`socialdb: V1__schema.sql, V2__seed_posts.sql` (1 publicación por usuario, referenciando los mismos UUIDs hijos)

Los 5 UUIDs viven como constantes literales en ambos scripts y quedan documentados en el README junto con la explicación de D5.

5. Contratos de API

auth-service (:8081)

Método	Ruta	Auth	Request	Response
POST	<code>/api/auth/login</code>	—	<code>{username, password}</code>	<code>{accessToken, expiresIn, user: {id, alias, firstName, lastName}}</code>
GET	<code>/api/users/me</code>	Bearer	—	<code>{id, username, firstName, lastName, birthDate, alias}</code>

social-service (:8082)

Método	Ruta	Auth	Request	Response
GET	<code>/api/posts?page=&size=</code>	Bearer	—	<code>{content: PostView[], page, size, totalElements}</code>
POST	<code>/api/posts</code>	Bearer	<code>{message}</code>	<code>PostView (201)</code>
POST	<code>/api/posts/{id}/likes</code>	Bearer	—	<code>{postId, likeCount, likedByMe: true}</code>
DELETE	<code>/api/posts/{id}/likes</code>	Bearer	—	<code>{postId, likeCount, likedByMe: false}</code>

`PostView: {id, message, createdAt, authorId, authorAlias, likeCount, likedByMe, isOwn}`

`authorId`, `authorAlias` y `createdAt` los fija siempre el servidor a partir del JWT y del reloj; nunca se aceptan del cliente.

WebSocket

`ws://localhost:8082/ws/likes?token=<jwt>`

Broadcast a todas las sesiones cuando cambia un contador:

```
{ "type": "LIKE_UPDATED", "postId": "...", "likeCount": 7 }
```

El token viaja como query param porque el `WebSocket` del navegador no permite headers en el handshake. Se valida en un `HandshakeInterceptor` que rechaza la conexión si el JWT es inválido. Trade-off documentado: en producción se usaría un ticket de un solo uso emitido por el endpoint HTTP, para que el JWT no aparezca en logs de acceso ni en el historial del navegador.

Errores

`@RestControllerAdvice` en ambos servicios devolviendo **RFC 7807 Problem Details**:

```
{
  "type": "https://periferia.social/errors/self-like-not-allowed",
  "title": "No puedes dar like a tu propia publicación",
  "status": 409,
  "detail": "...",
  "correlationId": "..."
}
```

Maapeo: validación → 400, credenciales inválidas → 401, regla de dominio violada → 409, recurso inexistente → 404, resto → 500 sin filtrar el stacktrace.

Documentación

springdoc-openapi con `springdoc.swagger-ui.path=/docs` en ambos servicios, según pide el enunciado.

6. Diseño de dominio

Las reglas de negocio viven en las entidades, no en los servicios de aplicación ni en la base de datos. Esto es lo que sustituye a los procedimientos almacenados (D7) y lo que hace que los tests más valiosos no necesiten Spring.

Agregado `Post` :

`message` no vacío tras `trim`, máximo 280 caracteres → `InvalidPostMessageException`

`createdAt` lo asigna el dominio en la construcción; nunca se acepta del cliente

`like(userId)` :

- si `userId == authorId` → `SelfLikeNotAllowedException` (409)

- si el usuario ya dio like → `DuplicateLikeException` (409)

- en caso contrario registra el like e incrementa `likeCount` en la misma transacción

`unlike(userId)` : si no existe el like → operación idempotente, sin error

Agregado `User` (auth-service):

`alias` y `username` no vacíos y normalizados

`birthDate` en el pasado

La contraseña nunca se expone; el hash no sale del agregado

La restricción `UNIQUE(post_id, user_id)` es una red de seguridad frente a concurrencia, no la sede de la regla.

7. `packages/core` — reutilización web/móvil

Es la pieza que el enunciado evalúa con más atención. Todo lo que no sea renderizado vive aquí.

```
packages/core/src/
├── api/
│   ├── httpClient.ts      # fetch nativo; inyecta Authorization y X-Correlation-Id
│   ├── authApi.ts
│   ├── postsApi.ts
│   └── realtimeClient.ts  # WebSocket con reconexión y backoff
├── domain/
│   ├── types.ts          # Post, User, PostView, LikeEvent
│   └── schemas.ts        # Zod: validación compartida web/móvil
├── store/
│   └── authStore.ts      # Zustand: token, usuario, login/logout
└── hooks/
    ├── useLogin.ts
    ├── useProfile.ts
    ├── usePosts.ts
    ├── useCreatePost.ts
    ├── useLikePost.ts    # update optimista
    └── useRealtimeLikes.ts # evento WS → reconcilia el cache de TanStack Query
```

Restricción dura: `core` no importa nada de DOM ni de React Native. Se usa `fetch` nativo (existe en ambas plataformas) en lugar de axios, y `WebSocket` nativo.

Costura entre plataformas — una sola función:

```
configureCore({
  apiBaseUrl: string,
  wsBaseUrl: string,
  storage: { getItem, setItem, removeItem }, // async
})
```

Se llama una vez al arrancar cada app. La web inyecta un adaptador sobre `localStorage`; el móvil, uno sobre `expo-secure-store`. **Nada más difiere entre plataformas.**

Estado: Zustand para la sesión (token + usuario autenticado) y TanStack Query para el estado de servidor (feed, perfil). Es una de las dos combinaciones que el enunciado autoriza expresamente.

Interacción optimista + tiempo real: `useLikePost` aplica el cambio en el cache antes de la respuesta y revierte si falla. `useRealtimeLikes` escucha el WebSocket y escribe el `likeCount` autoritativo en el cache de TanStack Query. Ambos coexisten porque el evento del servidor siempre gana. Este código se escribe una vez y corre idéntico en web y móvil.

8. Aplicaciones

apps/web (React + TypeScript)

Vite + react-router + Tailwind.

Login — usuario y contraseña, error visible, redirección al feed

Feed — composer arriba, lista descendente; cada tarjeta muestra alias, mensaje, fecha, contador de likes y botón; las propias van marcadas y con el botón deshabilitado; el contador se actualiza solo al llegar el evento WS

Perfil — nombres, apellidos, fecha de nacimiento y alias del autenticado

Ruta protegida: sin token válido, redirección a login

apps/mobile (React Native / Expo)

react-navigation + expo-secure-store. Alcance reducido según el enunciado.

Login

Feed con like y actualización en tiempo real

Solo `View`, `Text`, `Pressable`, `FlatList` y `StyleSheet`. **Cero lógica de negocio:** todos los hooks y el cliente de API se importan de `core`. Expo Web queda habilitado para poder abrir la app en un navegador sin necesidad de Xcode ni del SDK de Android.

9. Observabilidad

Actuator en ambos servicios: `/actuator/health` (con check de conectividad a BD), `/actuator/info`, `/actuator/prometheus`

Micrometer con métricas de negocio además de las de plataforma: `posts_created_total`, `likes_total`, `websocket_sessions_active`

Logs JSON estructurados (logstash-logback-encoder) con `correlationId` en MDC. Un filtro servlet lee el header `X-Correlation-Id` o genera uno; `core/api/httpClient` lo envía desde el front; el `@RestControllerAdvice` lo devuelve en cada error. Permite seguir una petición desde el clic del usuario hasta el log del servicio.

Prometheus en el compose, con scrape de ambos servicios

Grafana con datasource y un **dashboard provisionado** por ficheros: tasa de peticiones, latencia p95, likes por minuto, sesiones WebSocket activas

Healthchecks reales en `docker-compose` con `depends_on: {condition: service_healthy}`, no esperas fijas

10. Pruebas

Nivel	Herramienta	Qué cubre
Dominio	JUnit 5	Reglas de la sección 6: self-like, doble like, mensaje inválido, unlike idempotente. Sin Spring, milisegundos.
Controladores	@WebMvcTest + MockMvc	Contratos, códigos de estado, formato Problem Details, rechazo sin token
Integración	Testcontainers (1 por servicio)	auth: login real contra Postgres real. social: crear post → like → contador correcto y evento emitido.
Core (TS)	Vitest	useLikePost optimista con rollback; reconciliación de useRealtimeLikes

La pirámide es deliberadamente ancha en la base: las reglas de negocio viven en el dominio (sección 6), así que los tests más valiosos no necesitan levantar Spring ni la base de datos y corren en milisegundos. Testcontainers se reserva para verificar lo único que no se puede simular con fidelidad: el comportamiento real de Postgres frente a la restricción `UNIQUE(post_id, user_id)` bajo concurrencia.

11. Entrega

```
docker compose up
```

Levanta seis contenedores: `postgres` (con ambas bases creadas por script de init), `auth-service`, `social-service`, `web`, `prometheus` y `grafana`. La aplicación queda usable en <http://localhost:5173> sin instalar Java, Node ni pnpm.

El enunciado solo exige dockerizar los microservicios y la base de datos. Se incluyó también la web porque el coste es bajo —un build multi-stage que compila `packages/core` y sirve el resultado estático con nginx— y elimina por completo la fricción de puesta en marcha para quien evalúe.

La app móvil se sirve como Expo Web en `:5174`, etiquetada como tal. El simulador de iOS no se puede contenedorizar —solo corre en macOS bajo Xcode— pero `react-native-web` sí permite compilar la misma capa de vista a DOM. Se incluyó porque ahorra a quien evalúa instalar Xcode y descargar un simulador solo para comprobar la reutilización del core; y se etiqueta explícitamente como «no es la app nativa» para no dar una impresión falsa. Para verla como aplicación nativa: `pnpm --filter mobile ios`.

CORS

Ambos servicios declaran explícitamente los orígenes permitidos (`security.cors.allowed-origins`) en lugar de usar comodín, porque se permite la cabecera `Authorization` y conviene saber exactamente quién puede enviarla.

Este punto es un aviso que merece quedar escrito: **la ausencia de CORS es invisible desde `curl`**, porque la política la aplica el navegador y no el servidor. La API respondía `200` sin las cabeceras y toda verificación por línea de comandos pasaba, mientras que la aplicación web habría fallado

silenciosamente al no poder leer ninguna respuesta. Hay un test de preflight (`CorsConfigTest`) precisamente para que no vuelva a pasar desapercibido.

Nota sobre la configuración de la web: Vite compila las variables `VITE_*` dentro del bundle, así que las URLs del backend se fijan como `ARG` en tiempo de construcción. Apuntan a `localhost` porque quien resuelve esas direcciones es el navegador del evaluador, no el contenedor; los nombres internos de la red de Docker no resolverían desde fuera. Para un despliegue real se serviría la configuración en tiempo de ejecución, por ejemplo con un `config.js` generado al arrancar.

Artefactos

1. **Repositorio público en GitHub** — monorepo completo
2. `README.md` — instalación en un comando, credenciales de prueba, URLs de Swagger/Grafana, y la **sección de trade-offs**: microservicios vs. monolito modular, por qué no hay procedimientos almacenados, bases separadas y consistencia eventual, token en el query param del WebSocket, feed completo vs. literal, y **"qué haría con más tiempo"** (arquitectura event-driven para likes, proyección de autores vía eventos, API gateway, rate limiting, refresh tokens, tracing distribuido)
3. `docs/ARQUITECTURA.md` — fuente del PDF exigido, exportado a PDF
4. **Scripts de BD** — las migraciones Flyway con los usuarios predefinidos
5. **Video demo** — recorrido por `docker compose up` , Swagger, la web (login → perfil → publicar → like), dos ventanas en paralelo mostrando el like propagarse en tiempo real, la app en el simulador de iOS, y el dashboard de Grafana

Plan de ejecución

Bloque	Trabajo
Día 1 mañana	Monorepo, docker-compose, Postgres + Flyway + seed, <code>auth-service</code> completo con Swagger y tests
Día 1 tarde	<code>social-service</code> : posts, likes, WebSocket, observabilidad, tests
Día 2 mañana	<code>packages/core</code> + <code>apps/web</code> completa
Día 2 tarde	<code>apps/mobile</code> , Grafana, README de trade-offs, PDF y video

12. Fuera de alcance (declarado)

Se excluyen deliberadamente y se documentan como evolución, no como olvido: registro de usuarios, refresh tokens, seguir/dejar de seguir, comentarios, imágenes, notificaciones, paginación infinita, API gateway, service discovery, tracing distribuido, CI/CD y despliegue en la nube.